

Discrete Flow Matching Summary

Harivansh Luchmun

March 8, 2026

1 Discrete Flow Matching (DFM) Summary

1.1 Introduction

From the ideas of Conditional Flow Matching, Discrete Flow Matching was developed [2]. Instead of working with continuous values as we do in CFM, we are working with discrete values. DFM can be used in the context of taking in arbitrary discrete values and creating things that make sense. One good example is that it can be used to generate new text from random sequences of words. As letters and words cannot take continuous values, we can use DFM instead of CFM.

The main idea is to sample a sequence of tokens from a noise distribution and transition those tokens in the sequence so that the sequence looks like it came from a sequence from the target distribution. The tokens in the sequence transition based on the probability that the token is incorrect, which the model determines because it has learned what intermediate sequences should look like at time t . If we took a sentence, for example, then the initial input would be a sentence that contains a bunch of characters that may or may not make words. The intermediate states would begin to look like words because the model is fairly confident that some of these words should be correct based on its training data. The final state would ideally be a clear coherent sentence with understandable words.

They state that DFM can be useful because it achieved strong evaluation metrics [2]. They trained a 1.7B parameter model achieving a 6.7% score on Pass@1 and a score of 13.4% on Pass@10. Pass@1 measures how correct the output is likely to be on the first attempt and Pass@10 measures how likely there are to be correct outputs in ten outputs. The model also got a low generative perplexity score of 9.7 which could indicate it can potentially be a source of competition for autoregressive models. The authors also stated that there are many improvements that could be done to DFM to have better performance in the future [2].

1.2 Setup and Notation

We follow a similar process as Flow Matching in Discrete Flow Matching. We need to find a way to transition between random noise samples to a coherent output. They define a sequence x that is an array of elements or tokens $(x^1, x^2, \dots, x^N)$. In the sequence, each token can take a value within a max word size d , where $[d] = \{1, \dots, d\}$. What this means is that

there is a sequence of tokens where tokens are at most d in size. The max space for all possible sequences is defined as $D = [d]^N$. They also define a Probability Mass Function (PMF) that represents probabilities for sequences. For example, we take $N = 2$ and $d = \{0, 1, 2\}$ then we have possible values sequences such as $(0, 1), (2, 1), \dots$ that have an assigned probability based on how they appear in the dataset. This PMF is what is changing over time because initially it is noise then it approximates the target sequences.

The PMF that they define is a delta function $\delta_y, y \in D$.

$$\delta_y(x) = \prod_{i=1}^N \delta_{y_i}(x_i)$$

$$\delta_{y_i}(x_i) = 1 : x_i = y_i$$

$$\delta_{y_i}(x_i) = 0 : x_i \neq y_i$$

The delta function evaluates if the sequence is correct in approximating the target x . Each token in the sequence is multiplied and each token evaluates to 1 if that token matches the token in the sequence for the target x . Essentially this delta function describes if the sequence x is equal to the target sequence y .

There are many ways to pair source and target data together. The first and most simplest way is just to sample from each distribution directly.

$$(X_0, X_1) \sim p(X_0) q(X_1)$$

Another way to find the source distribution p is to create a mask token and to have the source distribution be a sequence of mask tokens $p(x) = \delta_m(x)$ where the starting point is always the same and $x^i = [MASK]$. Another way to create the source distribution is to draw each x_i for each token from a uniform distribution ranging across D and each value is $[d]$, resulting in many random tokens as the starting point, but in this paper they choose to focus on the masking method for tokenization.

Then they introduce two forms of coupling π for creating an (X_0, X_1) pair. The first is called unconditional coupling (U-coupling) which is independent coupling $\pi(x_0, x_1) = p(x_0)q(x_1)$. X_1 is sampled from the training set and X_0 is a sequence of mask tokens.

$$(X_0, X_1) = ((m, \dots, m), X_1)$$

The second choice of coupling is called conditional coupling, where $\pi(x_0, x_1) = p(x_0 | x_1)q(x_1)$. Which is more clearly defined as:

$$(X_0, X_1) = (I \odot X_1 + (1 - I) \odot (m, \dots, m), X_1)$$

In this form of coupling, the target sequence is still sampled from the training set, but the source sequence is the same as the target sequence but some of the tokens are mask tokens because $I \in 0, 1$ is a random variable and the entry wise product allows us to take a random number of mask tokens paired with target tokens.

In order to move from the source sequence to the target sequence, they implement something similar to FM by defining a marginal probability path that also uses a form of interpolation from p to q

$$p_t(x) = \sum_{x_0, x_1 \in \mathcal{D}} p_t(x \mid x_0, x_1) \pi(x_0, x_1)$$

where

$$p_t(x \mid x_0, x_1) = \prod_{i=1}^n p_t(x_i \mid x_0, x_1)$$

We have seen something similar in Flow Matching [1], where $p_t(x \mid x_0, x_1)$ is based on the time and x comes from an encoding of information between the two distributions. In this case we define the probability of a sequence at a certain time as the multiplication of the probabilities of tokens at different positions of the sequence at that time step. This means that the PMF on this sequence at this time would be based on $\pi(x_0, x_1)$ and the corresponding probabilities from the tokens for that given sequence. The conditions they laid out are $p_0(x_i \mid x_0, x_1) = \delta_{x_0}(x_i)$ and $p_1(x_i \mid x_0, x_1) = \delta_{x_1}(x_i)$ meaning that at time $t = 0$ we are at the source sequence and at time $t = 1$ we are at the target sequence because both of these should evaluate to 1 from the delta function which will also satisfy the PMF requirement.

They consider the conditional probability path as a sum of m conditional probabilities

$$p_t(x_i \mid x_0, x_1) = \sum_j \kappa_{i,j}^t w_j(x_i \mid x_0, x_1),$$

All individual weights must sum to 1, $\sum_j \kappa_{i,j}^t = 1$ and $\kappa_{i,j}^t \geq 0$ which defines the scheduler. This scheduler applies a value to each token in the sequence which is based on the time and weights can vary by position. They also propose a variation of linear interpolation to match the CFM path.

$$p_t(x_i \mid x_0, x_1) = (1 - \kappa_t) \delta_{x_0}(x_i) + \kappa_t \delta_{x_1}(x_i)$$

At time $t = 0$, $\kappa_0 = 0$ and at time $t = 1$, $\kappa_1 = 1$ and the time increases monotonically. They also propose another variation of linear interpolation with the scheduler that adds a uniform noise that is based on a probability at different times. This is useful because the distribution can look different at different times allowing for the model to explore more paths.

$$p_t(x_i \mid x_0, x_1) = \kappa_t^1 \delta_{x_1}(x_i) + \kappa_t^2 p_u(x_i) + \kappa_t^3 \delta_{x_0}(x_i)$$

κ_t^1 and κ_t^3 both must equal 0 in order to satisfy the boundary conditions of having the initial mask sequence at $t = 0$ and the target sequence at $t = 1$

1.3 Continuous-Time Discrete Markov Chains

In order to follow FM in the discrete scenario, they have to find a way to generate velocity for a given sample X and time t . To do this they use a Continuous-Time discrete Markov Chain. X_t will essentially transition between different possible sequences in D based on the velocity field $u_t^i(X_t), i \in N$.

$$X_{t+h}^i \sim \delta_{X_t^i}(\cdot) + h u_t^i(\cdot, X_t)$$

The probability velocity is similar to the velocity in Flow Matching. In this equation, to transition to different tokens in another sequence each token is assigned a probability and updated independently.

In order to generate the probability velocity they take the key insight from Flow Matching (Lipman et al. (2023)). u_t can be generated from marginalizing over the conditional velocities. In **Theorem 2** of Discrete Flow Matching [2], they state that the given conditional probability velocity that generates a conditional probability path can be turned into the marginal velocity and defined as

$$u_t^i(x_i, z) = \sum_{x_0, x_1} u_t^i(x_i, z \mid x_0, x_1) p_t(x_0, x_1 \mid z)$$

which generates the desired marginal probability path and by Bayes Rule,

$$p_t(x_0, x_1 \mid z) = \frac{p_t(z \mid x_0, x_1) \pi(x_0, x_1)}{p_t(z)}$$

They define $u_t^i(x_i, z)$ as the conditional probability velocity which determines how the sequence should change from z_i to a x_i at a time t and serves as a transition velocity. What this means is that there are many such z sequences at this time and we want to transition to the next state x but $u_t^i(x_i, z)$ evaluates token i and fixes the rest of the sequence so that z and x differ at most by one token. They take this average velocity for this token similar to flow matching, by factoring in how likely this $z \sim \pi(x_0, x_1)$ and the associated transition rate with that because you would know how to move from the source to target sequence. $u_t^i(x_i, z)$ must be greater than 0 to assign it a probability and it is not possible for $x_i = z_i$.

1.4 Continuity Equation

The Continuity Equation describes the state probability rate $\dot{p}_t(x)$, $x \in D$ for the discrete case. They model the continuity equation as a partial differential equation

$$\dot{p}_t(x) + \text{div}_x(p_t u_t) = 0$$

and a divergence equation for a discrete case is defined as

$$\text{div}_x(v) = \sum_{z \in D} [v(z, x) - v(x, z)]$$

The divergence calculated represents the total flux coming in and out of the system. $v(z, x)$ defines the flux $x \rightarrow z$ and $v(x, z)$ defines the flux $z \rightarrow x$, basically calculating the rate from transitioning to and from the state x . The probability flux at a state involves all sequences with at most one token difference from x . Essentially this gives us the transition probability rate. It means that if the divergence is positive we have a negative transition rate whereas if there is less divergence going then the transition rate to that state is positive. Which makes sense because if you have lots of divergence in that area you want to transition that token to another token that has lower outgoing flux. Applying this to FM for discrete cases the divergence equation becomes

$$\text{div}_x(p_t u_t) = - \sum_{z \in D} p_t(z) \sum_{i=1}^N \delta_z(\bar{x}^i) u_t^i(x_i, z)$$

In this equation we sum over all possible sequences from the current state, but only keeps ones that differ by at most one token in the sequence as we can see with $\delta_z(\bar{x}^i)$. The equation also assigns probability to the changing of one coordinate in the token with $u_t^i(x_i, z)$

We can rearrange the equation to see this more clearly.

$$\dot{p}_t(x) = -\text{div}_x(p_t u_t)$$

$$\dot{p}_t(x) = \sum_{z \in D} p_t(z) \sum_{i=1}^N \delta_z(\bar{x}^i) u_t^i(x_i, z)$$

To calculate the marginal probability velocity to generate p_t they define

$$u_t^i(x_i, z) = \sum_{j=1}^m a_t^{i,j} \hat{w}_t^j(x_i, z) + b_t^{i,j} \delta_z(x_i)$$

where,

$$w_t^j(x_i, z) = \sum_{x_0, x_1 \in \mathcal{D}} w_j(x_i | x_0, x_1) p_t(x_0, x_1 | z)$$

$u_t^i(x_i, z)$ calculates the transition rate of a token given this current z . $a_t^{i,j} = \dot{\kappa}_t^{i,j} - \kappa_t^{i,j} \frac{\dot{\kappa}_t^{i,\ell}}{\kappa_t^{i,\ell}}$ and $b_t^{i,j} = \frac{\dot{\kappa}_t^{i,\ell}}{\kappa_t^{i,\ell}}$ applies a scalar weight that changes with time which is based on the scheduler. $w_t^j(x_i, z)$ essentially calculates the weight of how likely the current token came from an x_0 and x_1 sequence as the posterior probability. The w basis can help determine how close to stay towards the source distribution or the target distribution and paired with the scheduler this can tell us how fast to move at certain times. For example we may want to stay closer to the source distribution earlier on so we may want a slower scheduler and a larger w^1 .

1.5 Sampling Backwards

In addition to just sampling forward in time to generate p_{t+h} , they add the idea of sampling backwards in time. Simply changing coefficients of $a_t^{i,j}$ and $b_t^{i,j}$.

$$u_t^i(x_i, z) = \frac{\dot{\kappa}_t}{\kappa_t} \left(\delta_z(x_i) - p_{0|t}(x_i | z) \right)$$

They calculate the velocity to sample backwards by factoring in the noise given from the scheduler $\frac{\dot{\kappa}_t}{\kappa_t}$, the current state and the prediction from the denoiser portion $p_{0|t}(x_i | z)$ which calculates how likely a certain token x_i belonged in this sequence. So we can move from target to source by using the current token and a noise posterior instead of the current token and clean posterior.

1.6 Corrector Sampling

Another form of sampling that allows for noise to be introduced at intermediate states is corrector sampling. Corrector sampling involves both forward and backward sampling. There is a variation where $\alpha_t - \beta_t = 1$ which makes steps move forward in time with $1 + \beta_t$

and move backwards with $-\beta_t$. Another variation is where $\alpha_t - \beta_t = 0$ which allows for corrector iterations. This is important because from **Theorem 4** they state that $\alpha_t - \beta_t = 1$ provides a probability velocity that generates p_t and $\alpha_t - \beta_t = 0$ with $t \in [0, 1]$, and small h is guaranteed to converge to a sample from p_t

1.7 Experiments

They conducted experiments involving code generation, language modeling, and surprisingly image generation. They used HumanEval, MBPP to evaluate their model for coding tasks. They created a model with 1.7 billion parameters and performed both unconditional and conditional couplings for training. For language modeling they found that C-coupling proved to be effective for conditional tasks and that DFM is closing the gap on autoregressive models.

For the coding evaluation they found that when conditioned on long solutions and non-trivial code generations the model performed well.

For image generation the authors also used CIFAR-10 as their dataset. They used DFM in an interesting way. They set the sequence length $N = 32 \times 32 \times 3$ and for $[d] = 256$ to represent the rgb value of a pixel. They used FID to evaluate the quality of the images generated and achieved a score of 3.6 FID for 1024 NFE [2] which is the amount of function evaluations or model uses.

1.8 Conclusion

Discrete Flow Matching offers a unique way to apply the ideas of Continuous Flow Matching in a discrete space. For language and code generation this novel idea could eventually be competition for autoregressive models currently in use. The authors claim that there is much room for work in the field of DFM and that there are improvements to be made [2].

Sources

[1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” International Conference on Learning Representations (ICLR), 2023. [Online]. Available: <https://arxiv.org/abs/2210.02747>

[2] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. Chen, G. Synnaeve, Y. Adi, Y. Lipman, and M. Fair, “Discrete Flow Matching,” Advances in Neural Information Processing Systems (NeurIPS), 2024. [Online]. Available: <https://arxiv.org/abs/2402.XXXXX>